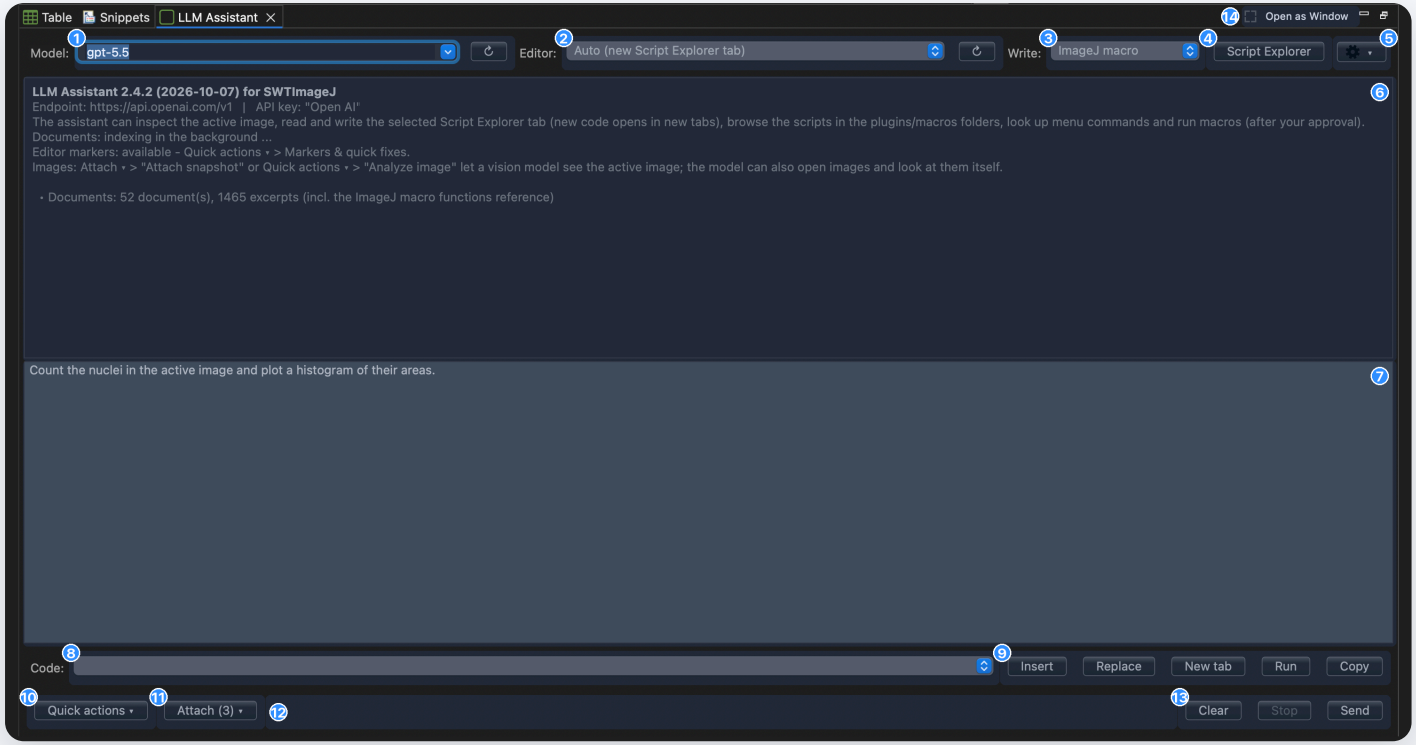
LLM Assistant for SWTImageJ

Reference card · a language model that reads and writes your scripts, inspects images and runs code

Version 2.4.2 · OpenAI · LM Studio · Ollama · OpenAI-compatible



Open the chat

Plugins › LLM Assistant › Chat…

In **SWTImageJ** the chat opens as a window. Applications that embed SWTImageJ can dock it as a view – shown here docked in Bio7 (dark theme), with the same controls.

Ask in plain language – e.g. *"Count the nuclei in the active image and plot a histogram of their areas."* The assistant writes the code, runs it after your approval and fixes errors itself.

The numbers refer to the list below.

The chat window at a glance

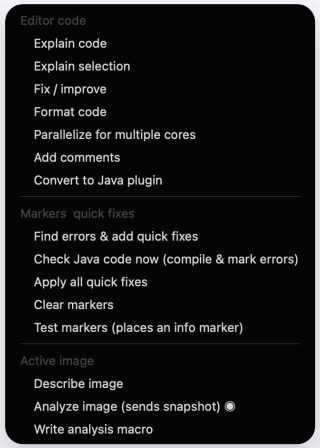
1. **Model** – choose or type a model; ↻ fetches the list from the server.
2. **Editor** – target for reading and writing code; *Auto* = selected Script Explorer tab.
3. **Write** – language: ImageJ macro, Java, JavaScript, BeanShell, Python.
4. **Script Explorer** – opens the Script Explorer; new code opens in new tabs.
5. ⚙ **menu** – *General Preferences…* and *Documents…*
6. **Transcript** – conversation, tool calls and the documents used.
7. **Message** – `Enter` sends, `Shift` + `Enter` new line.
8. **Code** – all code blocks of the conversation.
9. **Insert · Replace · New tab · Run · Copy** – use the selected block; a failed run puts the error into the message box.
10. **Quick actions** ▾ – one-click prompts (page 2).
11. **Attach** ▾ – what is sent with each message (page 2).
12. **Activity** – spinner with phase and time, e.g. *Waiting for model… 12 s*.
13. **Clear · Stop · Send** – new conversation · cancel the request or macro · send.
14. **Docked view** – when docked (e.g. in Bio7): *Open as Window* undocks, `F2` fullscreen.

AI Actions, tools & setup

What the assistant does for you – and what the model may call on its own

Version 2.4.2 · Plugins › LLM Assistant

Quick actions



Editor code
Explain, fix, format, parallelize, comment – or **convert to a Java plugin**.

Markers & quick fixes
Error markers with fixes in the editor; **double-click** marked text to apply. The last four items work without the model.

Active image
Describe, **analyze** (◉ sends a snapshot) or write an analysis macro.

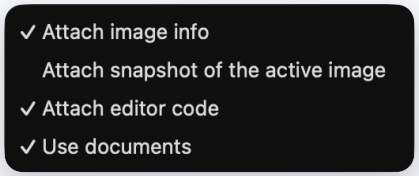
Java in the assistant

- *Write* › *Java*: compiled in memory and run; errors go back to the model, which fixes and retries.
- Statements, a `PlugIn` / `PlugInFilter` or a `main` class.
- Plugins are installed into `plugins/LLM_Plugins` (needs a JDK).

Tips

- From a macro: `call("llmassistant.ChatWindow.ask", "…");`
- Disable *Tool use* for models without function calling.
- The welcome text warns if the plugin is installed twice.

Attach – sent with each message



- **Image info** – size, type, calibration, ROI, statistics.
- **Snapshot** – the active image as displayed, max. 1024 px (vision model).
- **Editor code** – the code of the target editor.
- **Use documents** – best-matching excerpts of your documents.

Defaults: ⚙ › *General Preferences…*

Reference documents · ⚙ › Documents…

- Files or folders (drag & drop): text, Markdown, HTML, code, `.docx`, `.odt`, PDF.
- Built in: the **ImageJ macro functions reference**.
- Offline keyword search, optional **hybrid search** (embeddings); **test search** in the dialog.
- Tick a source folder to let the model grep and read it.

Privacy – what goes to the server

- Messages, attached image info / editor code, tool results.
- Snapshots only when attached or via `view_image`; switch off *Allow image snapshots* to prevent any upload.
- Only matching document excerpts, never whole files.
- Local servers (LM Studio, Ollama) keep everything on your computer.

Tools the model can call

CONTEXT	`get_imagej_context` `get_active_image_info` `get_log` `get_results_table` `search_commands`
IMAGES	`view_image` `open_image` `select_image` `list_sample_images`
EDITOR	`get_editor_text` `set_editor_text` `open_script` `list_scripts` `read_script`
RUN	`run_code` `run_editor` `check_code` `save_java_plugin`
MARKERS	`mark_issues` `apply_quick_fixes` `clear_markers`
KNOWLEDGE	`search_documents` `list_documents` `search_source` `read_source_file` `find_symbol`

Running code and replacing the whole editor ask for your approval (*Confirmations* in the preferences).

Setup in four steps

1. Copy `LLM_Assistant.jar` into the ImageJ `plugins` folder and restart (or **Help › Refresh Menus**).
2. **Plugins › LLM Assistant › General Preferences…**: add an API key (empty for local servers) and the server URL.
3. **Fetch models**, choose one and **Test connection**.
4. Open **Plugins › LLM Assistant › Chat…** and ask.

OpenAI `https://api.openai.com/v1`
LM Studio `http://127.0.0.1:1234/v1`
Ollama `http://localhost:11434/v1`
Keys: `LLM_Assistant.properties`